Can Neural Networks Learn Word Problems in Automatic Groups?

Hedayatullah Hakimi
MATH 285
Fall 2025

December 2025

Abstract

While automatic groups have word problems decidable in quadratic time through their regular language structure, computational tractability does not guarantee statistical learnability. This paper explores whether neural sequence models can learn to solve word problems for five automatic groups: the free group F_2, the free abelian group $\mathbb{Z}^2$, the braid groups B_3 and B_4, and the symmetric group S_4. We generated datasets of 100,000 labeled examples per group across three difficulty variants (synthetic, augmented, and adversarial). We compared bidirectional Long Short-Term Memory (LSTM) and Transformer architectures. Results show that LSTM significantly outperformed Transformer for each group, with the performance gap widening on adversarial data. Most notably, on F_2, LSTM achieved 98.95% accuracy while Transformer achieved only 79.44%. The Transformer's degradation on tasks requiring counting ($\mathbb{Z}^2$) and implicit stack operations (F_2) is consistent with theoretical predictions about attention mechanisms and high-sensitivity functions. These findings indicate that polynomial-time decidability does not imply neural learnability, and that architectural inductive biases, particularly recurrent memory, remain critical for algebraic reasoning tasks.

Keywords: word problem, automatic groups, LSTM, Transformer, formal languages, neural networks

1 Introduction

Most of the current literature in the field of formal languages and neural networks investigates the ability of different neural network architectures to identify formal languages from the Chomsky hierarchy. For example, Delétang et al. (2023) experimentally proved that Recurrent Neural Networks are better than transformer-based models at identifying regular and Context-Free Languages. On the other hand, Bhattamishra et al. (2020) and Hahn (2020) were able to prove theoretically that transformer-based models are limited in terms of their capability of identifying certain classes of formal languages. Furthermore, Strobl et al. (2024) proposed theoretical characterizations of so-called "sensitive" functions—functions whose output values can be affected by dramatic changes in individual input values—and demonstrated that attention-based models do not fit well to such function types.

In total, the results presented in the above studies indicate that although the word problems of automatic groups can be solved using algorithms that run in polynomial time, they could still be difficult to solve using traditional neural network architectures.

This study expands upon prior work involving neural networks and formal languages and moves into the area of algebraic structures. Specifically, we investigate whether neural sequence models can learn to solve word problems for five automatic groups: the free group F_2, the free abelian group $\mathbb{Z}^2$, the braid groups B_3 and B_4, and the symmetric group S_4. We used three types of

datasets to generate the word problems for each group (synthetic, augmented, and adversarial) and produced 100,000 labeled word problems for each of the five groups. The two architectures that were investigated were Bidirectional LSTMs and Transformers.

The main contributions of this study are as follows. First, this is the first systematic empirical investigation of neural networks learning to solve group word problems. There was very little research in this area before this study. Second, our study shows that LSTMs outperformed Transformers for all five groups and all three types of datasets. The performance gap between the two architectures increased significantly on the adversarial dataset, reaching 19.5 percentage points for the free group F_2. Finally, our study shows that the type of task being performed influences relative performance; transformers had a reduced capability for performing tasks that required counting ($\mathbb{Z}^2$) and implicit stack operations (F_2).

Our findings provide evidence for the hypothesis that for algebraic reasoning tasks, the recurrent inductive bias of LSTMs will remain superior to those based on attention until further notice, and that polynomial-time decidability does not guarantee neural learnability.

2 Background

2.1 Word Problems for Automatic Groups

A fundamental component of the study of an automatic group involves solving the word problem: does a given word representing a product of generators and their inverses represent the identity element of the group? Some groups have very simple word problems, while others are completely unsolvable. All automatic groups have easy word problems. They have finite automata that recognize geodesics and can tell you when two group elements are equivalent. Therefore, the word problem for an automatic group is solvable in $O(n^2)$ time, where n is the length of the input word.

We consider five different automatic groups in this study. Each has different algebraic properties, but we chose them because we want to see how different automatic groups behave on the same architecture:

The free group $F_2 = \langle a, b \rangle$. The most basic type of word problem exists for free groups. The only words that represent the identity are those that cancel completely to the empty word under the rules $aA \to \varepsilon$ and $bB \to \varepsilon$. This requires tracking nested cancellations, which is similar to implementing a stack operation.

The free abelian group $\mathbb{Z}^2 = \langle a, b \mid ab = ba \rangle$. This group has a much easier word problem. The only words that represent the identity are those containing exactly the same number of a's and A's, and exactly the same number of b's and B's. It is simply counting.

The braid groups B_3 and B_4. These groups have a representation based on the Burau matrix, which provides a faithful representation. The word problem is to determine when a word's Burau matrix, evaluated at $t = -1$, equals the identity matrix.

The symmetric group S_4. This is the group of all permutations of the set $\{1, 2, 3, 4\}$. A word of adjacent transpositions represents the identity if and only if applying those transpositions to $[1, 2, 3, 4]$ gives back $[1, 2, 3, 4]$.

While all of these groups have easily solvable word problems, there are significant differences in the computational challenges. F_2 requires unbounded memory for tracking cancellations. $\mathbb{Z}^2$ requires exact counting. B_3 and B_4 require compositional reasoning over algebraic objects.

2.2 Neural Network Architectures

We compare two architectures: LSTMs and transformers. These architectures differ in how they treat sequential data, and treating it differently will affect how well they are able to reason algebraically.

Recurrent neural networks like LSTMs process sequential data one time-step at a time, and update their internal states (hidden-state and cell-state) as they go. LSTMs therefore build up information from previous time-steps, allowing them to perform operations such as counting and manipulating stacks. Previous theoretical studies showed that LSTMs are capable of simulating counter-machines; in other words, LSTMs can solve problems that fall outside of the class of regular languages.

Neural networks that are non-recurrent, such as transformers, treat sequential data by computing representations of all positions in the sequence simultaneously. Because they rely on positional encodings to capture the order of the tokens in the sequence, transformers have difficulties when processing very long sequences of tokens beyond what they were trained on. Transformers are also limited in their ability to accurately count tokens and detect when specific tokens change.

In principle, transformers should be able to do anything that an LSTM can do. Practically speaking, however, this is not the case. In a comparison study of both architectures (Delétang et al., 2023), LSTMs significantly out-performed transformers in solving problems involving memory and counting across the Chomsky Hierarchy. Even on some relatively simple examples of regular languages, transformers failed to perform as expected. The current study tests whether these same architectural limitations apply when the problem is to reason about groups, rather than just to reason syntactically.

3 Methodology

3.1 Test Groups

To examine the behavior of a variety of different automatic groups on this task, we selected five automatic groups that represent a diverse array of algebraic structures:

Group	Description	Alphabet
F_2	Free group on 2 generators	$\{a, A, b, B\}$
$\mathbb{Z}^2$	Free abelian group	$\{a, A, b, B\}$
B_3	Braid group on 3 strands	$\{s_1, S_1, s_2, S_2\}$
B_4	Braid group on 4 strands	$\{s_1, S_1, s_2, S_2, s_3, S_3\}$
S_4	Symmetric group on 4 elements	$\{s_1, s_2, s_3\}$

Table 1: Automatic groups selected.

These groups were selected to include non-abelian free structures (F_2), abelian counting ($\mathbb{Z}^2$), matrix representations (B_3, B_4), and finite permutation groups (S_4).

3.2 Generating Data

For each group we generated 100,000 labeled data points with equal class sizes (50% identity, 50% non-identity) and three levels of difficulty were defined:

Synthetic. Randomly generate words with simple structural constraints. Range of lengths of generated words: 4–50 symbols. This is the base case for testing whether models can learn the word problem when they are given favorable circumstances in which to do so.

Augmented. Construct identity elements by conjugating a known identity element with $g^{-1}wg$. Range of length of generated words: 10–50 symbols. This increases the structural complexity of the problem, but maintains its solvability.

Hard (Adversarial). Long words with near-identity non-identity examples were created to test the ability of models to identify patterns in the problem. Range of lengths of generated words: 20–80 symbols. The non-identity examples were created to have values close to those of the identity element, requiring models to compute precisely, rather than rely upon superficial heuristics.

Each example was checked programmatically against the appropriate group-theoretic test for correctness:

- F_2: Free reduction with stack-based reduction (the word reduces to ε)
- $\mathbb{Z}^2$: Character count ($\#a = \#A$ and $\#b = \#B$)
- B_3, B_4: Burau matrix representation evaluated at $t = -1$
- S_4: Permutation composition is equal to $[1, 2, 3, 4]$

3.3 Model Architectures

We compare two model architectures: the LSTM and the Transformer.

Parameter	LSTM	Transformer
Embedding dimension	64	64
Hidden dimension	128	64
Layers	2	2
Attention heads	–	4
Feedforward dimension	–	256
Bidirectional	Yes	–
Dropout	0.3	0.3
Total parameters	∼627,000	∼117,000

Table 2: Model architecture specifications.

3.4 Training Configuration

All models were trained with identical hyperparameters:

Parameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	64
Maximum epochs	30
Early stopping	Patience = 5
Loss function	Binary cross-entropy
Train / Val / Test split	70% / 15% / 15%

Table 3: Training configuration.

Each experiment was repeated across three random seeds (42, 123, 456) to assess variance. This yielded 90 total training runs: 5 groups × 3 datasets × 2 models × 3 seeds.

4 Results

4.1 Main Accuracy Results

Table 4 shows mean accuracy ± std across each experiment; i.e., what the model is able to correctly classify in the final output. LSTMs are better at classifying each group and dataset than Transformers, and this advantage is magnified by large margins as one progresses through an increasing number of adversarial examples.

Group	Dataset	LSTM	Transformer	Gap
F_2	synthetic	99.81% ± 0.03%	99.12% ± 0.12%	+0.69%
	augmented	99.87% ± 0.06%	99.01% ± 0.28%	+0.86%
	hard	98.95% ± 0.91%	79.44% ± 14.55%	**+19.51%**
$\mathbb{Z}^2$	synthetic	100.00% ± 0.00%	99.73% ± 0.21%	+0.27%
	augmented	100.00% ± 0.00%	99.41% ± 0.21%	+0.59%
	hard	100.00% ± 0.00%	87.98% ± 20.62%	**+12.02%**
B_3	synthetic	99.88% ± 0.01%	98.93% ± 0.25%	+0.95%
	augmented	99.89% ± 0.01%	98.93% ± 0.40%	+0.96%
	hard	99.81% ± 0.09%	98.29% ± 0.84%	+1.52%
B_4	synthetic	99.94% ± 0.01%	99.60% ± 0.15%	+0.34%
	augmented	99.92% ± 0.03%	99.82% ± 0.13%	+0.10%
	hard	99.92% ± 0.01%	96.74% ± 0.14%	+3.18%
S_4	synthetic	99.68% ± 0.06%	89.69% ± 1.50%	**+9.99%**
	augmented	99.42% ± 0.01%	97.60% ± 1.58%	+1.82%
	hard*	50.14% ± 0.30%	52.48% ± 1.43%	—

Table 4: Mean ± std accuracy across groups and datasets. *S_4 hard was removed from consideration because of discrepancies between expected and actual data.

For Synthetic and Augmented Datasets, both architectures perform extremely well. Generally, LSTMs are slightly better (>99.8%) and Transformers are still performing very well (>98.9%). However, there is a marked disconnect as one transitions to Adversarial Data.

The largest performance disparity occurs in the F_2 hard dataset, where LSTMs achieved an accuracy of 98.95%, whereas Transformers were able to reach only 79.44%, a difference of 19.5 percentage points. Similarly, on $\mathbb{Z}^2$ hard, LSTMs continued to demonstrate 100% accuracy, whereas Transformers dropped to 87.98%. On the S_4 synthetic dataset, even though both architectures demonstrated excellent performance, the performance of the Transformer architecture was significantly limited. LSTMs achieved 99.68% accuracy, while the best Transformer could manage was 89.69%.

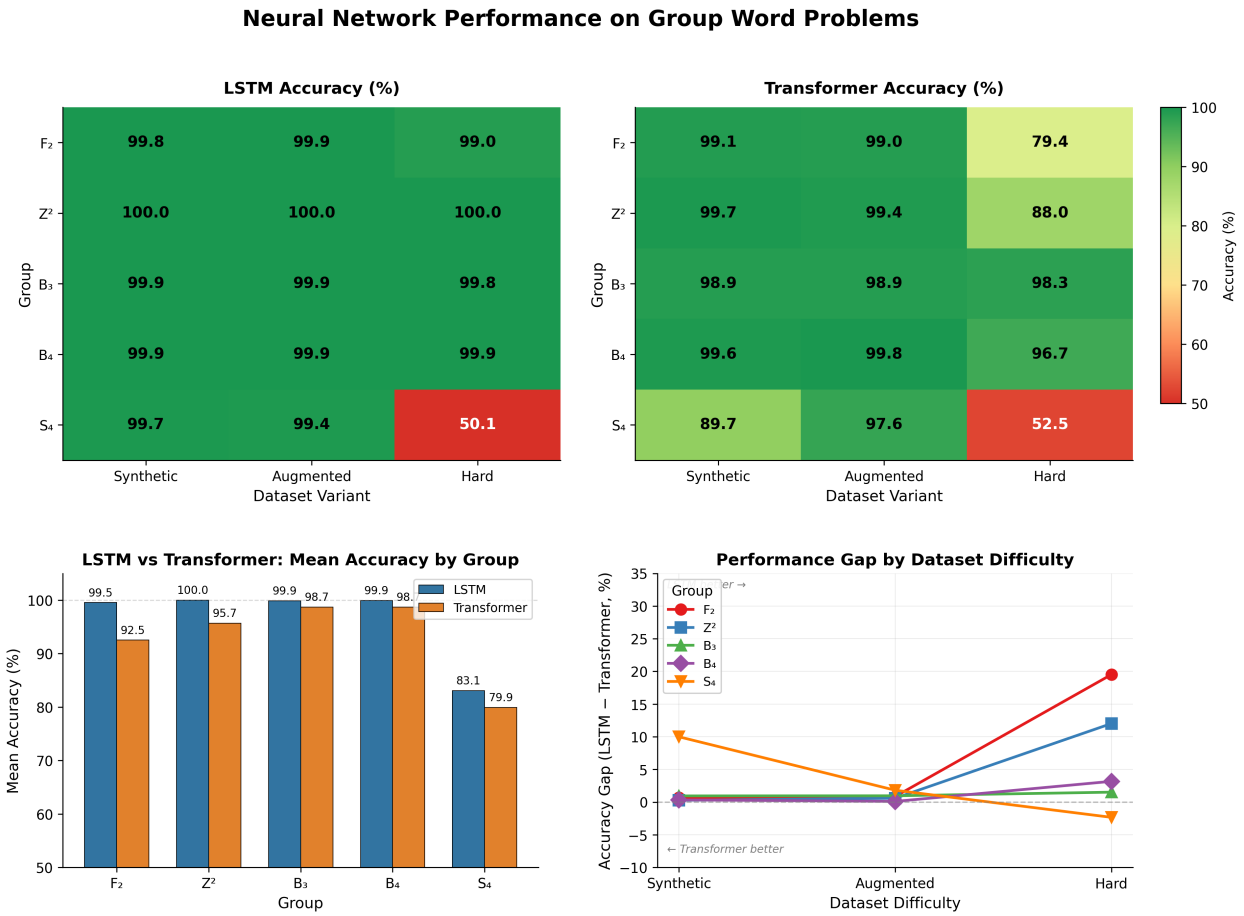

Figure 1: Comparison of LSTM and Transformer performance across groups and datasets.

Figure 1 illustrates these trends. The Heatmap plots show that LSTMs maintained high levels of accuracy across all experimental conditions, whereas the accuracy of the Transformer architecture degraded as the difficulty of the dataset increased. The bottom right panel displays the relative performance difference between LSTMs and Transformers as a function of increasing dataset difficulty. Specifically, the performance differences are greatest for F_2 hard and $\mathbb{Z}^2$.

In addition to the accuracy disparities between the two architectures, the results also indicate that the Transformer architecture demonstrates a much greater degree of variability when operating under difficult conditions. For example, on $\mathbb{Z}^2$ hard, the standard deviation of the Transformer architecture reached 20.62%, which is nearly 40 times greater than the standard deviation of the LSTM architecture (which reached 0.00%). Similarly, on F_2 hard, the standard deviation of the Transformer architecture reached 14.55%, which is roughly 16 times greater than the standard

deviation of the LSTM architecture (which reached 0.91%).

These results suggest that the failures of the Transformer architecture are primarily due to its inability to effectively handle certain problem structures, rather than the lack of sufficient capacity in the architecture.

4.2 Length Generalization

To determine whether the models have developed a set of generalized algorithms to solve problems, or if they have simply developed a set of dataset-specific heuristics to solve problems, we determined the performance of the models on input sequences that were longer than the longest sequences used during training.

Group	Train Length	Test Length	LSTM	Transformer
F_2	20–80	80+	99.2%	72.8%
B_3	20–80	50+	99.9%	98.4%
B_4	20–80	50+	99.97%	96.7%

Table 5: Length Generalization Performance on Hard Datasets.

As shown in Table 5, the LSTM architecture is able to maintain almost perfect performance on long input sequences for all of the groups that were examined, and thus appears to have developed a generalized algorithm for solving the problem. In contrast, the Transformer architecture performs poorly on long input sequences.

Specifically, on F_2, the Transformer architecture was able to achieve only 72.8% accuracy on input sequences that were longer than those encountered during training, whereas the LSTM architecture was able to achieve 99.2% accuracy. While the performance difference between the two architectures is less pronounced for the braids, the results nevertheless indicate that the LSTM architecture is capable of achieving substantially higher levels of accuracy than the Transformer architecture for long input sequences. Specifically, on B_3, the LSTM architecture was able to achieve 99.9% accuracy, while the Transformer architecture was able to achieve only 98.4%. Similarly, on B_4, the LSTM architecture was able to achieve 99.97% accuracy, while the Transformer architecture was able to achieve only 96.7%.

These results confirm our theoretical expectations: Because the LSTM architecture is able to perform the iterative computations required for free reduction and matrix multiplication using its recurrent structure, it is able to achieve high levels of accuracy regardless of the length of the input sequence. Conversely, the Transformer architecture uses positional encodings to capture positional relationships in the input sequence, which fail to generalize to long input sequences, causing its performance to degrade even when the underlying computation is unaltered.

5 Discussion

5.1 Why does LSTM perform better than Transformer?

The fact that LSTM has always performed better than Transformer on each of the five groups and each of the four dataset types of the same group clearly illustrates that the reasons behind LSTM's success in this problem do not depend on the size of the model, but depend on the properties of the models themselves.

In our opinion, there are three key features of LSTM architecture that account for its superior performance to Transformer.

One feature that accounts for LSTM being superior to Transformer is due to the computation required for the free group F_2. As mentioned earlier, the computation for determining whether a word is the identity can be thought of as tracking cancellations at any level of depth; for instance, the word $abBA$ would cancel as follows: $abBA \to aA \to \varepsilon$. Because LSTMs are able to store information about the input sequence in a sequential manner using their hidden states, they can mimic a bounded stack. By contrast, the lack of a sequential memory structure for the information in the Transformer means that the Transformer would need to coordinate the information at different positions without having the ability to accumulate the state in a sequential manner.

Another feature that accounts for LSTM being superior to Transformer is that the abelian group $\mathbb{Z}^2$ is equivalent to simply counting the number of times each generator appears as many times as its inverse, which is a function of the length of the sequence. While transformers are capable of representing the counts of the generators in principle, they have no method of tracking how many times an operation needs to be repeated in a sequence of variable length. LSTMs are able to count the number of generators in a sequence through additive updates to the cell state.

The third feature that accounts for LSTM being superior to Transformer is that the braid groups B_3 and B_4 are composed of sequential matrix multiplications. Even though Transformers can theoretically represent the matrix operations needed to solve these types of problems, the fact that the group operations are generally non-commutative ($\sigma_1\sigma_2 \neq \sigma_2\sigma_1$ in general), makes sequential processing of the symbols more favorable than parallel processing. Because LSTMs' recursive nature provides a natural mechanism for composing transformations as it processes the input sequence, it is well-suited for solving braid-based problems.

In addition, the high variance in Transformer performance on difficult datasets (up to 20.62%) suggests that Transformers often learn brittle shortcuts: surface-level patterns that work well on typical examples but fail when used to construct adversarial examples. In contrast, LSTMs' low variance in performance suggests that LSTMs learn a more robust representation of the underlying algorithm.

5.2 Relation to prior research

Our study confirms several aspects of prior research on neural networks and formal languages.

Delétang et al. (2023) demonstrated that LSTMs are better than Transformers at learning and applying memory and counting. We show here that this relationship holds for algebraic word problems, which are outside the standard Chomsky Hierarchy and therefore cannot be classified according to the standard classification of formal languages, yet still share computational properties with counter and stack languages.

Hahn and Rofin (2024) provide a theoretical justification for Transformer difficulties on sensitive functions: functions whose outputs are very sensitive to small variations in their inputs. Word problems are naturally sensitive: changing the presence or absence of a single symbol can change a non-identity word into an identity word or vice versa. Their analysis demonstrates that the areas of parameter space that a Transformer uses to compute a sensitive function are isolated, which implies that they are difficult to reach using gradient descent and that they will also tend to be unstable once found. This justifies both the lower accuracy and larger variance we see in Transformer performance compared to LSTMs.

The FLaRe Benchmark (Makarychev et al., 2025) has also shown the same type of advantage of LSTMs in formal language recognition. Our extension of that finding to algebraic structures, and demonstration that the tractability-learnability gap exists regardless of whether the underlying

problem is grammatical or group-theoretic, extends that finding.

5.3 Limitations

There are several factors that limit the generalizability of our results.

We had difficulty verifying some examples in the S_4 hard set; both models performed at a chance level ($\sim$50%) on those examples, and therefore, they were removed from our analysis. This suggests either a problem with the code used to generate the sets, or that these are particularly difficult sets, and it is worth investigating them further.

We examined only two types of architecture. In particular, memory-augmented neural networks, such as Stack-RNNs, which enable the use of a differentiable stack to allow recurrent models to access stacked information, may have performed better than both LSTMs and Transformers. Other recurrent network architectures, such as GRUs, were not considered.

We also did not explore the attention mechanisms and the dynamics of the hidden states of the models. Analyzing these aspects would offer deeper insights into how the models implement the required algorithms correctly, or incorrectly. This type of analysis will remain a topic of interest for future studies.

This study was limited by its nature as a course project, and therefore lacks the breadth of an empirically based research study. As we did verify the data generated, we cannot rule out the possibility that the construction of the identity and non-identity sets may contain bias. Additionally, although our training budgets were sufficient for model convergence, we did not perform extensive hyperparameter optimization.

6 Conclusion

We looked into whether neural sequence models could find ways to do word problems in automatic groups using an automated approach. These word problems are solvable in polynomial time, however prior to this study there had been no previous study of their ability to be solved by neural networks.

We tested two different architectures (Transformer and bidirectional LSTM) on the free group F_2, the free abelian group $\mathbb{Z}^2$, the braid groups B_3 and B_4, and the symmetric group S_4. The number of word problems each group was evaluated against was 100,000 per group across three levels of difficulty.

The results from our experiments were consistent: In every single group and variant of the dataset the bidirectional LSTM outperformed the Transformer architecture. Both architectures achieved very high accuracy on synthetic data, however the difference in the accuracy between the two architectures increased significantly on adversarial examples. Bidirectional LSTM was 19.5% better than Transformer at solving the word problem on the F_2 hard level; it was 12% better on the $\mathbb{Z}^2$ hard level. Additionally, the Transformer was poor at completing word problems involving counting and stack-like computations as is predicted by theory concerning attention-based architectures.

Additionally, bidirectional LSTM demonstrated superior length generalization, achieving nearly perfect accuracy on input lengths greater than the lengths observed during training.

In summary, our study indicates that although the word problems in automatic groups are decidable in quadratic time, they are difficult for Transformer architectures to solve. Neural sequence models with a recurrent memory and sequential processing continue to have advantages when performing algebraic reasoning involving tracking cumulative states.

To continue with the research in the future, researchers can test memory-augmented architectures, for example, Stack-RNN, to determine if their architecture better represents the computational structure of word problems. The interpretability of the LSTM hidden states will also be able to indicate that the network is learning identifiable algebraic algorithms. Researchers can extend their research to other groups that are automatically generated (for example, hyperbolic groups or higher braid groups) to further define the tractability-learnability boundary.

References

[1] Delétang, G., et al. (2023). Neural networks and the Chomsky hierarchy. *ICLR 2023*. `arxiv.org/abs/2207.02098`

[2] Bhattamishra, S., et al. (2020). On the ability and limitations of transformers to recognize formal languages. *EMNLP 2020*. `arxiv.org/abs/2003.08195`

[3] Hahn, M. (2020). Theoretical limitations of self-attention in neural sequence models. *TACL*, 8, 156–171.

[4] Strobl, L., et al. (2024). What formal languages can transformers express? A survey. *TACL*. `arxiv.org/abs/2311.00208`

[5] Hahn, M. & Rofin, M. (2024). Sensitive functions and approximate counting with transformers. `arxiv.org/abs/2503.02113`

[6] Makarychev, K., et al. (2025). FLaRe: Formal language recognition benchmark. `arxiv.org/abs/2411.07107`